Lab Sheet 5: SQL Joins

INNER, LEFT, RIGHT, FULL, CROSS, NATURAL and SELF Joins - SQL Answers with Expected Outputs (MySQL 8.x)

Tables and sample data follow the lab sheet. Question 22 is not present in the lab sheet, so numbering goes from 21 to 23.

Setup (run once before Question 1)

```
CREATE DATABASE Lab5_Joins;
USE Lab5_Joins;

CREATE TABLE Department (DeptID INT PRIMARY KEY, DeptName VARCHAR(50));
CREATE TABLE Course (CourseID INT PRIMARY KEY, CourseName VARCHAR(50), Fees DECIMAL(10,2));
CREATE TABLE Student (Student_ID INT PRIMARY KEY, Name VARCHAR(50), DeptID INT, CourseID INT);
CREATE TABLE Employee (EmpID INT PRIMARY KEY, Name VARCHAR(50), DeptID INT,
                       Salary DECIMAL(10,2), ManagerID INT);

INSERT INTO Department VALUES (10, 'Computer Science'), (20, 'Mechanical'), (30, 'Electrical');
INSERT INTO Course VALUES (101, 'DBMS', 5000), (102, 'Data Structures', 4500);
INSERT INTO Student VALUES (1, 'Amit', 10, 101), (2, 'Rahul', 20, 102), (3, 'Sita', NULL, 101);
INSERT INTO Employee VALUES (201, 'John', 10, 60000, 202),
                            (202, 'Sara', 30, 75000, NULL),
                            (203, 'Mike', 10, 50000, 202);
```

INNER JOIN

1. Demonstrate an INNER JOIN between Student and Department.

SQL Answer:

```
SELECT * FROM Student S
INNER JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

Student_ID	Name	DeptID	CourseID	DeptID	DeptName
1	Amit	10	101	10	Computer Science
2	Rahul	20	102	20	Mechanical

Note: Sita has no department (DeptID is NULL), so she does not appear in an INNER JOIN.

2. Display student names along with their department names using INNER JOIN.

SQL Answer:

```
SELECT S.Name AS StudentName, D.DeptName
FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

StudentName	DeptName
Amit	Computer Science
Rahul	Mechanical

3. Display employee names along with their department names.

SQL Answer:

```
SELECT E.Name AS EmployeeName, D.DeptName
FROM Employee E INNER JOIN Department D ON E.DeptID = D.DeptID;
```

Expected Output:

EmployeeName	DeptName
John	Computer Science
Sara	Electrical
Mike	Computer Science

Note: Without ORDER BY, the row order may differ slightly on your screen. The rows themselves are the same.

4. Display students along with their course details.

SQL Answer:

```
SELECT S.Name AS StudentName, C.CourseName, C.Fees
FROM Student S INNER JOIN Course C ON S.CourseID = C.CourseID;
```

Expected Output:

StudentName	CourseName	Fees
Amit	DBMS	5000.00
Rahul	Data Structures	4500.00
Sita	DBMS	5000.00

5. Find students belonging to a particular department using INNER JOIN.

SQL Answer:

```
SELECT S.Name, D.DeptName
FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID
WHERE D.DeptName = 'Computer Science';
```

Expected Output:

Name	DeptName
Amit	Computer Science

LEFT and RIGHT OUTER JOIN

6. Demonstrate a LEFT OUTER JOIN between Student and Department.

SQL Answer:

```
SELECT S.Name, D.DeptName
FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

Name	DeptName
Amit	Computer Science
Rahul	Mechanical
Sita	NULL

Note: LEFT JOIN keeps every row of the left table (Student).

7. Display all students, including those who do not have a matching department.

SQL Answer:

```
SELECT S.Student_ID, S.Name, D.DeptName
FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

Student_ID	Name	DeptName
1	Amit	Computer Science
2	Rahul	Mechanical
3	Sita	NULL

8. Demonstrate a RIGHT OUTER JOIN between Student and Department.

SQL Answer:

```
SELECT S.Name, D.DeptName
FROM Student S RIGHT JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

Name	DeptName
Amit	Computer Science
Rahul	Mechanical
NULL	Electrical

Note: RIGHT JOIN keeps every row of the right table (Department).

9. Display all departments, including departments having no students.

SQL Answer:

```
SELECT D.DeptID, D.DeptName, S.Name AS StudentName
FROM Student S RIGHT JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

DeptID	DeptName	StudentName
10	Computer Science	Amit
20	Mechanical	Rahul
30	Electrical	NULL

10. Find students who are not assigned to any department using LEFT JOIN.

SQL Answer:

```
SELECT S.Student_ID, S.Name
FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID
WHERE S.DeptID IS NULL;
```

Expected Output:

Student_ID	Name
3	Sita

11. Find departments having no students using RIGHT/LEFT JOIN.

SQL Answer:

```
SELECT D.DeptID, D.DeptName
FROM Department D LEFT JOIN Student S ON D.DeptID = S.DeptID
WHERE S.Student_ID IS NULL;
```

Expected Output:

DeptID	DeptName
30	Electrical

12. Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround.

SQL Answer:

```
SELECT S.Name AS StudentName, D.DeptName
FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID
UNION
SELECT S.Name AS StudentName, D.DeptName
FROM Student S RIGHT JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

StudentName	DeptName
Amit	Computer Science
Rahul	Mechanical
Sita	NULL
NULL	Electrical

Note: MySQL has no FULL OUTER JOIN, so LEFT JOIN UNION RIGHT JOIN is used.

FULL OUTER JOIN, CROSS JOIN and NATURAL JOIN

13. Demonstrate a CROSS JOIN between Student and Course.

SQL Answer:

```
SELECT S.Name, C.CourseName
FROM Student S CROSS JOIN Course C;
```

Expected Output:

Name	CourseName
Amit	DBMS
Amit	Data Structures
Rahul	DBMS
Rahul	Data Structures
Sita	DBMS

Sita	Data Structures

14. Calculate the number of possible Student-Course combinations using CROSS JOIN.

SQL Answer:

```
SELECT COUNT(*) AS TotalCombinations
FROM Student S CROSS JOIN Course C;
```

Expected Output:

TotalCombinations
6

Note: 3 students x 2 courses = 6 combinations.

15. Demonstrate a NATURAL JOIN between two suitable tables.

SQL Answer:

```
SELECT * FROM Student NATURAL JOIN Department;
```

Expected Output:

DeptID	Student_ID	Name	CourseID	DeptName
10	1	Amit	101	Computer Science
20	2	Rahul	102	Mechanical

Note: NATURAL JOIN matches on the common column (DeptID) and shows it only once.

16. Compare the result of NATURAL JOIN and INNER JOIN.

SQL Answer:

```
-- 1. NATURAL JOIN
SELECT * FROM Student NATURAL JOIN Department;
-- 2. INNER JOIN
SELECT * FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

NATURAL JOIN (DeptID appears once, first):

DeptID	Student_ID	Name	CourseID	DeptName
10	1	Amit	101	Computer Science
20	2	Rahul	102	Mechanical

INNER JOIN (DeptID appears twice):

Student_ID	Name	DeptID	CourseID	DeptID	DeptName
1	Amit	10	101	10	Computer Science
2	Rahul	20	102	20	Mechanical

Comparison:

Method	Behavior
NATURAL JOIN	Implicit match on common columns; removes the duplicate key column.
INNER JOIN	Explicit match using the ON clause; keeps both key columns unless you select columns.

SELF JOIN

17. Perform a SELF JOIN on an Employee table to show employee-manager relationships.

SQL Answer:

```
SELECT E1.Name AS Employee, E2.Name AS Manager
FROM Employee E1 LEFT JOIN Employee E2 ON E1.ManagerID = E2.EmpID;
```

Expected Output:

Employee	Manager
John	Sara
Sara	NULL
Mike	Sara

Note: Sara has no manager (ManagerID is NULL).

18. Display employees along with their managers using SELF JOIN.

SQL Answer:

```
SELECT E1.EmpID, E1.Name AS EmployeeName, E2.Name AS ManagerName
FROM Employee E1 INNER JOIN Employee E2 ON E1.ManagerID = E2.EmpID;
```

Expected Output:

EmpID	EmployeeName	ManagerName
201	John	Sara
203	Mike	Sara

Multiple Table Joins

19. Join three tables: Student, Department and Course.

SQL Answer:

```
SELECT S.Name, D.DeptName, C.CourseName
FROM Student S
INNER JOIN Department D ON S.DeptID = D.DeptID
INNER JOIN Course C ON S.CourseID = C.CourseID;
```

Expected Output:

Name	DeptName	CourseName
Amit	Computer Science	DBMS
Rahul	Mechanical	Data Structures

20. Display student name, course name and department name using multiple joins.

SQL Answer:

```
SELECT S.Name AS StudentName, C.CourseName, D.DeptName
FROM Student S
JOIN Course C ON S.CourseID = C.CourseID
JOIN Department D ON S.DeptID = D.DeptID;
```

Expected Output:

StudentName	CourseName	DeptName
Amit	DBMS	Computer Science
Rahul	Data Structures	Mechanical

21. Find employees whose salary is greater than the average salary of their department.

SQL Answer:

```
SELECT E.Name, E.Salary, E.DeptID
FROM Employee E
WHERE E.Salary > (SELECT AVG(Salary) FROM Employee WHERE DeptID = E.DeptID);
```

Expected Output:

Name	Salary	DeptID
John	60000.00	10

Note: Department 10 average is 55000, and only John (60000) is above it.

Joins with GROUP BY and Aggregates

23. Display departments and the total number of students in each department.

SQL Answer:

```
SELECT D.DeptName, COUNT(S.Student_ID) AS TotalStudents
FROM Department D LEFT JOIN Student S ON D.DeptID = S.DeptID
GROUP BY D.DeptID, D.DeptName;
```

Expected Output:

DeptName	TotalStudents
Computer Science	1
Mechanical	1

Electrical	0

24. Display courses along with the number of students enrolled in each course.

SQL Answer:

```
SELECT C.CourseName, COUNT(S.Student_ID) AS EnrolledStudents
FROM Course C LEFT JOIN Student S ON C.CourseID = S.CourseID
GROUP BY C.CourseID, C.CourseName;
```

Expected Output:

CourseName	EnrolledStudents
DBMS	2
Data Structures	1

25. Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions to generate a student-department-course report.

SQL Answer:

```
SELECT D.DeptName, C.CourseName,
       COUNT(S.Student_ID) AS TotalEnrolled,
       AVG(C.Fees) AS AvgFees
FROM Department D
INNER JOIN Student S ON D.DeptID = S.DeptID
LEFT JOIN Course C ON S.CourseID = C.CourseID
GROUP BY D.DeptName, C.CourseName;
```

Expected Output:

DeptName	CourseName	TotalEnrolled	AvgFees
Computer Science	DBMS	1	5000.000000
Mechanical	Data Structures	1	4500.000000

Note: The Student table has no Fees column, so AVG(C.Fees) from the Course table is used.